

DEMO PROJEK KELOMPOK 7 DAN 8

I Putu Evo Wira Saputra – 2401010250/Kelas C

Kelompok 7

Global adaptive thresholding

```
import cv2

img = cv2.imread('hunde.jpg', cv2.IMREAD_GRAYSCALE)

if img is None:
    print("Error: Could not find the image. Please check the
file name.")
else:
    thresh_val, otsu_thresh = cv2.threshold(img, 0, 255,
cv2.THRESH_BINARY + cv2.THRESH_OTSU)
    cv2.imshow('Original Grayscale', img)
    cv2.imshow('Automatic Otsu Thresholding Result',
otsu_thresh)

    print(f"Otsu automatically picked the threshold value:
{thresh_val}")

    cv2.waitKey(0)
    cv2.destroyAllWindows()
```

```
import cv2

# 1. Load the image in grayscale (black and white shades)
img = cv2.imread('hunde.jpg', cv2.IMREAD_GRAYSCALE)

if img is None:
    print("Error: Could not find the image. Please check the
file name.")
else:
    threshold_value = 127
    thresh_val, global_thresh = cv2.threshold(img,
threshold_value, 255, cv2.THRESH_BINARY)
    cv2.imshow('Original Grayscale', img)
    cv2.imshow('Strict Global Thresholding Result',
global_thresh)

    cv2.waitKey(0)
    cv2.destroyAllWindows()
```

Local Adaptive Thresholding

```
import os
import cv2
import matplotlib.pyplot as plt

# Mendapatkan path absolut direktori tempat skrip ini berada
script_dir = os.path.dirname(os.path.abspath(__file__))
image_path = os.path.join(script_dir, "contohgambar.jpeg")

# Membaca gambar asli
img_bgr = cv2.imread(image_path)

# Memastikan gambar berhasil dibaca
if img_bgr is None:
    raise FileNotFoundError(f"Gambar tidak ditemukan di path: {image_path}. Pastikan 'contohgambar.jpeg' ada di folder yang sama.")

# Mengonversi gambar ke Grayscale untuk proses thresholding
img_gray = cv2.cvtColor(img_bgr, cv2.COLOR_BGR2GRAY)

# --- THRESHOLDING LOKAL ADAPTIF (LOCAL ADAPTIVE) ---
# Menentukan parameter:
# block_size = Ukuran wilayah tetangga (harus bilangan ganjil, misal 11x11, 15x15)
# C = Konstanta yang dikurangi dari rata-rata/gaussian rata-rata (untuk fine-tuning)
block_size = 11
C = 0

# 1. Adaptive Mean Thresholding
thresh_adaptive_mean = cv2.adaptiveThreshold(
    img_gray, 255, cv2.ADAPTIVE_THRESH_MEAN_C,
    cv2.THRESH_BINARY, block_size, C
)

# 2. Adaptive Gaussian Thresholding (Memberi bobot Gaussian pada tetangga piksel)
thresh_adaptive_gaussian = cv2.adaptiveThreshold(
    img_gray, 255, cv2.ADAPTIVE_THRESH_GAUSSIAN_C,
    cv2.THRESH_BINARY, block_size, C
)

# Mengatur tampilan visualisasi premium (Horizontal 1x3 Grid)
plt.figure(figsize=(18, 6))
plt.suptitle(
    f"Pengaplikasian Local Adaptive Thresholding\n(Block Size = {block_size}x{block_size}, C = {C})",
    fontsize=16, fontweight='bold', y=0.98
)
```

```

# Panel 1: Gambar Asli Grayscale
plt.subplot(1, 3, 1)
plt.imshow(img_gray, cmap='gray')
plt.title("A. Gambar Grayscale Asli", fontsize=12,
fontweight='semibold')
plt.axis('off')

# Panel 2: Adaptive Mean Thresholding
plt.subplot(1, 3, 2)
plt.imshow(thresh_adaptive_mean, cmap='gray')
plt.title("B. Adaptive Mean Thresholding", fontsize=12,
fontweight='semibold')
plt.axis('off')

# Panel 3: Adaptive Gaussian Thresholding
plt.subplot(1, 3, 3)
plt.imshow(thresh_adaptive_gaussian, cmap='gray')
plt.title("C. Adaptive Gaussian Thresholding", fontsize=12,
fontweight='semibold')
plt.axis('off')

plt.tight_layout()
plt.show()

```

Kelompok 8

CCL dan KMEANS

```

import cv2
import numpy as np
import matplotlib.pyplot as plt

#
=====
# DEMO: Connected Component Labeling (CCL)
# Input: Sel Darah.jpeg
# Mata Kuliah: Pengolahan Citra Digital
#
=====

# --- 1. Baca gambar & konversi ke grayscale ---
citra_asli = cv2.imread('Sel_Darah.jpeg')
citra_asli_rgb = cv2.cvtColor(citra_asli, cv2.COLOR_BGR2RGB)
citra_gray = cv2.cvtColor(citra_asli, cv2.COLOR_BGR2GRAY)

# --- 2. Binarisasi: Otsu Thresholding ---
# Otsu otomatis mencari nilai T terbaik dari histogram
_, citra_biner = cv2.threshold(
    citra_gray, 0, 255,

```

```

        cv2.THRESH_BINARY + cv2.THRESH_OTSU
    )

    print("=" * 50)
    print(f"    Dimensi citra      : {citra_asli.shape}")
    print(f"    Dimensi biner      : {citra_biner.shape}")

    # --- 3. Terapkan CCL dengan statistik ---
    # connectedComponentsWithStats() mengembalikan:
    # - jumlah_label : total label (label 0 = background)
    # - matriks_label: setiap piksel berisi ID komponennya
    # - stats        : [x, y, w, h, area] per komponen
    # - centroids    : koordinat pusat massa (cx, cy) per
    # komponen
    jumlah_label, matriks_label, stats, centroids =
    cv2.connectedComponentsWithStats(
        citra_biner, connectivity=8
    )

    print(f"    Jumlah komponen   : {jumlah_label - 1} (label 0 =
    background)")
    print("=" * 50)

    # --- 4. Filter noise: hanya tampilkan area >= MIN_AREA ---
    MIN_AREA = 200
    objek_valid = [
        label for label in range(1, jumlah_label)
        if stats[label, cv2.CC_STAT_AREA] >= MIN_AREA
    ]

    print(f"{'Label':<8} {'Area (px)':<12} {'cx':<8} {'cy':<8}")
    print("-" * 40)
    for label in objek_valid[:10]: # Tampilkan max 10 objek
        terbesar
        area = stats[label, cv2.CC_STAT_AREA]
        cx, cy = centroids[label]
        print(f"    {label:<6} {area:<12} {cx:<8.1f} {cy:<8.1f}")
    print(f"    ... total {len(objek_valid)} objek valid (area >=
    {MIN_AREA} px)")
    print("=" * 50)

    # --- 5. Beri warna acak per komponen ---
    H, W = citra_biner.shape
    img_berwarna = np.zeros((H, W, 3), dtype=np.uint8)
    np.random.seed(42)
    for label in range(1, jumlah_label):
        warna = np.random.randint(80, 255, size=3).tolist()
        img_berwarna[matriks_label == label] = warna

    # --- 6. Gambar bounding box + centroid pada citra asli ---
    img_anotasi = citra_asli_rgb.copy()

```

```

for label in objek_valid:
    x = stats[label, cv2.CC_STAT_LEFT]
    y = stats[label, cv2.CC_STAT_TOP]
    w = stats[label, cv2.CC_STAT_WIDTH]
    h = stats[label, cv2.CC_STAT_HEIGHT]
    cx, cy = int(centroids[label][0]),
int(centroids[label][1])
    area = stats[label, cv2.CC_STAT_AREA]

    # Hanya anotasi objek yang cukup besar (supaya tidak
    penuh)
    if area >= 1000:
        cv2.rectangle(img_anotasi, (x, y), (x+w, y+h), (255,
255, 0), 1)
        cv2.circle(img_anotasi, (cx, cy), 4, (255, 50, 50), -
1)

# --- 7. Visualisasi ---
fig, axes = plt.subplots(1, 4, figsize=(16, 4))
fig.suptitle("Demo: Connected Component Labeling (CCL) - Sel
Darah", fontsize=12, fontweight='bold')

axes[0].imshow(citra_asli_rgb)
axes[0].set_title("1. Citra Asli")
axes[0].axis('off')

axes[1].imshow(citra_gray, cmap='gray')
axes[1].set_title("2. Grayscale")
axes[1].axis('off')

axes[2].imshow(citra_biner, cmap='gray')
axes[2].set_title(f"3. Citra Biner (Otsu)")
axes[2].axis('off')

axes[3].imshow(img_berwarna)
axes[3].set_title(f"4. Hasil CCL\n{jumlah_label - 1}
komponen")
axes[3].axis('off')

plt.tight_layout()
plt.savefig('ccl_SelDarah_output.png', dpi=150,
bbox_inches='tight')
plt.show()
print("  Output disimpan: Sel_Darah.jpeg")

```

```

import cv2
import numpy as np
import matplotlib.pyplot as plt
from sklearn.cluster import KMeans

```

```

#
=====
# DEMO: Segmentasi Berbasis K-Means Clustering
# Input: lenna.jpeg
# Mata Kuliah: Pengolahan Citra Digital
#
=====

# --- 1. Baca gambar & konversi BGR → RGB ---
citra_bgr = cv2.imread('lenna.png') # ← pastikan file
lenna.jpeg ada di folder yang sama
citra_asli = cv2.cvtColor(citra_bgr, cv2.COLOR_BGR2RGB)
print("=" * 50)
print(f" Dimensi citra      : {citra_asli.shape}")

# --- 2. Reshape: (H, W, 3) → (H*W, 3) ---
# Wajib sebelum input ke KMeans
# Setiap piksel menjadi 1 baris dengan nilai [R, G, B]
H, W, _ = citra_asli.shape
pixels = citra_asli.reshape((-1, 3)).astype(np.float32)
print(f" Setelah reshape   : {pixels.shape}")
print("=" * 50)

# --- 3. Elbow Method – cari K optimal (K=1..8) ---
print(" Menghitung Elbow Method...")
wcss_list = []
K_range = range(1, 9)
for k in K_range:
    km = KMeans(n_clusters=k, n_init=5, random_state=42,
max_iter=100)
    km.fit(pixels)
    wcss_list.append(km.inertia_)
    print(f"      K={k} → WCSS={km.inertia_:.0f}")
print("=" * 50)

# --- 4. Terapkan K-Means dengan beberapa nilai K ---
# Bandingkan hasil K=3, K=5, K=7
hasil_segmentasi = {}
for K in [3, 5, 7]:
    kmeans = KMeans(n_clusters=K, n_init=10, random_state=42)
    kmeans.fit(pixels)
    centers = kmeans.cluster_centers_.astype(np.uint8)
    labels = kmeans.labels_
    seg = centers[labels].reshape(H, W, 3)
    hasil_segmentasi[K] = seg
    print(f" K={K} selesai – {K} warna dominan")
print("=" * 50)

# --- 5. Visualisasi ---
fig, axes = plt.subplots(2, 3, figsize=(14, 9))

```

```

fig.suptitle("Demo: K-Means Clustering - Lenna", fontsize=13,
fontweight='bold') # ← judul sudah benar

# Baris atas: citra asli + elbow curve + K=3
axes[0, 0].imshow(citra_asli)
axes[0, 0].set_title("Citra Asli")
axes[0, 0].axis('off')

axes[0, 1].plot(list(K_range), wcss_list, marker='o',
color='purple', linewidth=2)
for k in [3, 5, 7]:
    axes[0, 1].axvline(x=k, linestyle='--', alpha=0.5,
label=f'K={k}')
axes[0, 1].set_xlabel('Jumlah Cluster K')
axes[0, 1].set_ylabel('WCSS (Inertia)')
axes[0, 1].set_title('Elbow Method')
axes[0, 1].legend(fontsize=8)
axes[0, 1].grid(True, alpha=0.3)

axes[0, 2].imshow(hasil_segmentasi[3])
axes[0, 2].set_title("K=3 (Kasar)")
axes[0, 2].axis('off')

# Baris bawah: K=5, K=7, palette warna
axes[1, 0].imshow(hasil_segmentasi[5])
axes[1, 0].set_title("K=5 (Sedang)")
axes[1, 0].axis('off')

axes[1, 1].imshow(hasil_segmentasi[7])
axes[1, 1].set_title("K=7 (Detail)")
axes[1, 1].axis('off')

# Panel terakhir: palette warna centroid K=7
kmeans7 = KMeans(n_clusters=7, n_init=10, random_state=42)
kmeans7.fit(pixels)
centers7 = kmeans7.cluster_centers_.astype(np.uint8)
palette = np.zeros((50, 7 * 50, 3), dtype=np.uint8)
for i, c in enumerate(centers7):
    palette[:, i*50:(i+1)*50] = c
axes[1, 2].imshow(palette)
axes[1, 2].set_title("Palette Warna K=7")
axes[1, 2].set_xticks([25 + i*50 for i in range(7)])
axes[1, 2].set_xticklabels([f"C{i}" for i in range(7)],
fontsize=8)
axes[1, 2].set_yticks([])

plt.tight_layout()
plt.savefig('kmeans_lenna_output.png', dpi=150,
bbox_inches='tight')
plt.show()
print(" Output disimpan: kmeans_lenna_output.png")

```

